MIDTERM STUDY GUIDE: ADD-ON

Introduction to IoT and Edge Computing · What the official guide leaves out

Read the official guide first. This sheet only adds what is in the slides but not in that guide.

Official guide: concepts and comparisons	This add-on: numbers, calculations, extra tables, practical code

Scope note

I compared the lecturer's study guide against the week 1, 4, 5 and 6 slides and the week 2–3 practical code. The guide is accurate; nothing in it needed correcting. The items below appear in the slides or practicals but are missing or only mentioned in passing. Weeks 2 and 3 have no slides in the repo, so for those weeks the official guide and the practical code are the only sources.

Important exam note

The official guide says you will not write full programs, but you may have to **read code and interpret output**. That makes the calculation (duty cycle), the matching exercises (wildcards) and the code tables below the most useful parts of this sheet.

Week 1 and Week 4 Additions

Device vs edge vs cloud (week 1)

	Device	Edge	Cloud
Role	Collects raw data	Processes data **near devices**	Large-scale storage and analytics
Resources	Very limited CPU and memory	Moderate	Effectively unlimited
Typical tasks	Read temperature, check battery, simple filtering	Event detection, aggregation, filtering	Model training, historical analysis, dashboards
Benefit	Low latency at the source	Less bandwidth, faster response	Store history, analyze trends

Placing tasks: read a temperature = **device**; immediate alarm = **device or edge**; one year of history = **cloud**; factory-wide trends = **cloud**.

Application priorities (week 1)

Application	Priority
Smart home	Convenience, cost
Healthcare	Reliability, privacy
Factory	Fast response, reliability
Agriculture	Long range, low power
Smart city	Scalability

Bandwidth vs data rate (week 4)

Bandwidth = the **maximum** amount transmittable in a given time. **Data rate** = the rate **actually** transferred. More bandwidth can support a higher data rate, but they are not the same thing. A temperature sensor sends 25–100 bytes every few minutes; a 1 Gbit/s link is about 125 MB/s.

Also: "range = 100 m" never means "always works at 100 m". Power, antenna, frequency, obstacles and interference all change it.

Radio states and duty cycle (week 4)

Radio state	Power demand
Transmit / receive	High
Idle / listening	Moderate
Sleep	Very low

Sleep most of the time	Wake briefly	Measure	Transmit a small message	Return to sleep

Duty cycle = active time ÷ total time. Energy for transmitting: **E ≈ I × t**.

	Device A	Device B
Behavior	Transmit 1 s every minute	Transmit 1 s every hour
Duty cycle	1 ÷ 60 = **1.67 %**	1 ÷ 3600 = **0.028 %**
Transmissions per day	60 × 24 = 1440	24
Transmit time per day	1440 s = 0.4 h	24 s = 0.00667 h
Energy at 30 mA	30 × 0.4 = **12 mAh/day**	30 × 0.00667 = **0.20 mAh/day**

High-yield
Device A uses about **60 times** the transmit energy of device B. Lower duty cycle → less radio-on time → longer battery life. More data also means more radio-on time. This is the main worked calculation in the slides and the official guide skips it, so be able to redo it with other numbers.

RFID, NFC and PoE (week 4)

Feature	Barcode	QR code	RFID
Reading method	Optical	Optical	**Radio wave**
Line of sight	Required	Required	**Not required**
Multiple items	One at a time	One at a time	**Multiple tags possible**
Cost	Very low	Very low	Higher
Typical use	Product ID	Links, tickets	Inventory, assets

Passive RFID tags can work without a battery. NFC is the centimeter-range relative (contactless payment, transit passes). **PoE** (Power over Ethernet) delivers **data and power over one Ethernet cable**: cameras, access points, gateways.

Cellular vs non-cellular LPWAN (week 4)

Cellular LPWAN	Non-cellular LPWAN
Licensed spectrum, run by telecom operators	**Unlicensed** open radio bands
NB-IoT, LTE-M	LoRa
Public network only, subscription	**Private gateway possible**

	NB-IoT	LTE-M	LoRa
Data	Lower	Higher	Very low
Mobility	Limited	**Better**	Limited
Infrastructure	Operator network	Operator network	Private or local gateway possible
Typical application	**Meters**	**Trackers**	**Farm and environmental sensors**

A real LoRa path: device → LoRa/LoRaWAN → gateway → Ethernet, cellular or Wi-Fi → Internet.

Week 5 Additions

6LoWPAN: three functions

Function	Detail
Header compression	Reduces the standard **40-byte IPv6 header to as few as 2–7 bytes**
Fragmentation and reassembly	Splits large IPv6 packets into fragments and rebuilds them at the receiver
Mesh routing adaptation	Supports routing across low-power wireless **mesh** topologies

The official guide lists the first two. The **border router** joins the mesh to the IPv6 Internet; past it the packet is ordinary IPv6. IPv6 solves the address shortage, **not** the limits of constrained devices.

Transport layer in one line

Network layer = host-to-host. Transport layer = process-to-process. Ports to know: **80 = HTTP**, **1883 = MQTT**.

TCP three-way handshake

SYN: connection request	SYN-ACK: received and accepted	ACK: connection confirmed

The handshake sets up **state** at both ends and agrees **sequence numbers** for ordered delivery. The ACK number is the **next expected byte**.

UDP: four things it does not do

Connectionless	No handshake
Unreliable delivery	Does not track lost, delayed or duplicated packets
No ordering	Packets can arrive in a different order
No flow control	Sends as fast as the application produces data

MQTT wildcard matching

The broker has these topics:

```
factory/machine01/temperature
factory/machine01/vibration
factory/machine02/temperature
factory/machine02/status
office/room101/temperature
```

Subscription	Receives
`factory/+/temperature`	`factory/machine01/temperature`, `factory/machine02/temperature`
`factory/machine01/+`	`factory/machine01/temperature`, `factory/machine01/vibration`
`factory/#`	All four `factory/...` topics, **not** `office/room101/temperature`

Trap

`factory/+/temperature` does **not** match `factory/line1/machine01/temperature`: `+` is exactly one level, and that topic has one level too many. `factory/#` also matches `factory` itself (zero remaining levels).

Why it matters: 500 machines × 3 measurements = 1,500 topics. A temperature dashboard needs **one** subscription instead of 500, and machine 501 is picked up automatically.

MQTT QoS message exchanges

QoS	Guarantee	Messages	Example
0	At most once	PUBLISH	Temperature telemetry (the next reading arrives soon anyway)
1	At least once	PUBLISH → **PUBACK**	Fire alarm, machine fault
2	Exactly once	PUBLISH → **PUBREC** → **PUBREL** → **PUBCOMP**	Critical control message, financial transaction

HTTP vs CoAP vs MQTT in one table

Question	HTTP	CoAP	MQTT
Main model	Request-response	Request-response	**Publish-subscribe**
Common transport	TCP	**UDP**	TCP
Architecture	Client-server	Client-server / resource	**Broker-based**
Strength	Web and API integration	Constrained resources	Telemetry and event distribution
Typical IoT example	Cloud REST API	Constrained sensor	Continuous sensor messaging

They are **not mutually exclusive**: one system may use more than one. **MQTT does not require JSON**; the payload is application-defined bytes.

Week 6 Additions

Stream vs datagram

TCP = stream	UDP = datagram
A continuous byte stream	Each `sendto()` creates a separate datagram
Does not preserve the boundaries of individual sends	**Message boundaries are preserved**
Send `"Hello"` then `"World"` → may arrive as `"HelloWorld"`	Send `"Message A"` then `"Message B"` → two datagrams

Analogy from the slides: TCP = sealed courier delivery; UDP = a postcard.

Which layer is each socket argument?

Python argument	Concept	Layer
`AF_INET`	IPv4 (`AF_INET6` for IPv6)	**Network**
`SOCK_STREAM`	TCP	**Transport**
`SOCK_DGRAM`	UDP	**Transport**

`accept()` **blocks** until a client connects. A socket transmits **bytes, not strings**, which is why `encode()` comes before sending.

Mosquitto commands

Command	Meaning
`mosquitto -v`	Start the broker and print connection and message activity
`mosquitto_sub -h localhost -t "test/topic" -v`	Subscribe to a topic on the local broker
`mosquitto_pub -h localhost -t "test/topic" -m "Hello IoT"`	Publish a message to that topic

`-h` = broker host, `-t` = topic, `-m` = message payload. Expected subscriber output: `test/topic Hello IoT`.

Run order

Broker: mosquitto -v	Subscriber	Sensor publisher	Edge processor and actuator

Start the **subscriber before the publisher**: a normal (non-retained) message published before anyone subscribes is not delivered later.

Multiple-subscriber experiment

Run `mqtt_subscriber.py` in two terminals and `mqtt_sensor.py` once. **Both subscribers receive the same message with no change to the publisher code.** Explanation: the publisher sends to the broker and topic; the broker distributes to every matching subscriber. This is decoupling in practice.

Topics in the edge loop

The **actuator subscribes to the command topic**, not the sensor topic. The edge processor is the only program that is **both** subscriber (sensor topic) and publisher (command topic).

Week 2 and 3 Practical Code

The official guide explains these ideas but shows none of the code you wrote.

Week 2: moving average (window 3)

```
recent_readings.append(temperature)
if len(recent_readings) > 3:
    recent_readings.pop(0)

if len(recent_readings) >= 3:
    ma_3 = sum(recent_readings[-3:]) / 3
else:
    ma_3 = None
```

You see...	You should think...
`random.uniform(20, 30)`	A **virtual sensor** producing a simulated reading
`datetime.now()`	The **timestamp** that makes this time-series data
`recent_readings.pop(0)`	The **oldest** value leaves the window
`sum(...) / 3`	The moving average of the last three readings
`ma_3 = None` → prints "Waiting"	Fewer than three readings: the average is **not available yet**, and it is not replaced by 0
`time.sleep(1)`	The **sampling interval**: one reading per second

Hand trace with readings 22, 24, 29, 23: the first two outputs are "Waiting", then (22 + 24 + 29) ÷ 3 = **25.0**, then (24 + 29 + 23) ÷ 3 = **25.33**.

Week 3: cold-chain monitoring device

Read sensors	Validate	Determine status	Build nested record	Serialize to JSON	Compare payload sizes

Check	Question it answers	Example from the assignment
Validation (quality)	Is the value **physically possible**?	Temperature −20 to 50 °C, humidity 0–100 %, battery 0–100 % → `VALID` / `INVALID`
Status	Is the value **acceptable for the cargo**?	Temperature below 2 = `TOO_COLD`, 2–8 = `NORMAL`, above 8 = `TOO_WARM`; humidity 30–70 = `NORMAL`; battery below 20 = `LOW_BATTERY`

Do not confuse

A reading can be **valid but not normal**. 10 °C is a perfectly possible temperature (`VALID`) and still bad for the medicine (`TOO_WARM`). Validation checks the sensor; status checks the situation.

The nested record groups related values under parent keys:

```
record = {
    "device_id": "coldbox_07",
    "timestamp": "...",
    "measurements": {"temperature": 5.2, "humidity": 48.0, "battery": 76.0},
    "quality":      {"temperature": "VALID", "humidity": "VALID", "battery": "VALID"},
    "status":       {"temperature": "NORMAL", "humidity": "NORMAL", "battery": "NORMAL"}
}
```

`record["status"]["temperature"]` reads one nested value. `json.dumps(record)` turns the whole dictionary into JSON text, and its length is the **payload size**: field names and nesting make the payload bigger, which is the JSON vs binary trade-off from the official guide.

Extra Self-Check

1. Where would you place each task: reading a temperature, raising an immediate alarm, storing a year of history?
2. What is the difference between bandwidth and data rate?
3. A radio transmits for 2 s every 10 minutes. What is the duty cycle?
4. The same radio draws 40 mA while transmitting. How many mAh per day does transmitting cost?
5. Give three ways RFID differs from a barcode.
6. What is PoE?
7. Which LPWAN technologies use licensed spectrum, and which use unlicensed?
8. NB-IoT or LTE-M for a moving delivery tracker? Why?
9. What are the three functions of 6LoWPAN?
10. Name the three steps of the TCP handshake.
11. Does `factory/+/temperature` match `factory/line1/machine01/temperature`?
12. Which topics does `factory/machine01/+` match in the five-topic example?
13. Which MQTT messages are exchanged for QoS 1 and for QoS 2?
14. Send `"Hello"` then `"World"` over TCP. Why might the receiver read `"HelloWorld"`?
15. Which layer does `AF_INET` belong to? `SOCK_DGRAM`?
16. Why start the subscriber before the publisher?
17. Which topic does the virtual actuator subscribe to?
18. In the moving-average code, why are the first two outputs "Waiting"?
19. A cold-chain reading is 10 °C. What are its quality and its status?
20. Why is a nested JSON record larger than the three raw numbers it carries?

Extra Self-Check Answers

1. Device; device or edge; cloud.
2. Bandwidth is the maximum that can be transmitted in a given time; data rate is what is actually transferred.
3. 2 ÷ 600 = 0.33 %.
4. 144 transmissions per day × 2 s = 288 s = 0.08 h. 40 mA × 0.08 h = **3.2 mAh/day**.
5. Radio instead of optical reading; no line of sight needed; many tags can be read at once (and it costs more).
6. Power over Ethernet: data and electrical power over the same Ethernet cable.
7. Licensed: NB-IoT and LTE-M (operator-run). Unlicensed: LoRa.
8. LTE-M: it handles moving devices better and supports higher data rates. NB-IoT suits static, low-data devices such as meters.
9. Header compression (40 bytes to 2–7), fragmentation and reassembly, mesh routing adaptation.
10. SYN, SYN-ACK, ACK.
11. No. `+` matches exactly one level, and that topic has an extra level.
12. `factory/machine01/temperature` and `factory/machine01/vibration`.
13. QoS 1: PUBLISH, PUBACK. QoS 2: PUBLISH, PUBREC, PUBREL, PUBCOMP.
14. TCP is a byte stream and does not preserve the boundaries of individual sends.
15. `AF_INET`: network layer (IPv4). `SOCK_DGRAM`: transport layer (UDP).
16. A non-retained message published before anyone has subscribed is not delivered afterwards.
17. The **command** topic (FAN ON / FAN OFF), not the sensor topic.
18. A window of 3 needs three readings; until then the average is `None`, which is honest about missing data.
19. Quality `VALID` (it is physically possible); status `TOO_WARM` (above 8 °C).
20. JSON carries field names, quotes and structure as text, which adds overhead on top of the values.

End of Add-On